

CSCC11 A3

Qiushi Chen 1007622306

April 2024

Theory Question

1 Question 1: Bias-Variance Decomposition and trade off

In the course for Bayesian model selection, we discussed how model selection depends on model complexity (described in detail in the course notes in Section 12.3). We discussed how increasing model complexity results in reducing the error as measured by the model likelihood during training, while increasing the divergence between the posterior and prior. This phenomenon is also characterized by the so-called bias-variance trade-off in machine learning. The bias of a model represents the average difference between the predicted values and the true values. The variance of a model, on the other hand, measures the variability of the predicted values around their mean. As the complexity of the model increases, the variance will tend to increase and the bias will tend to decrease.

With this context, in terms of the mean-squared-error (MSE) for the model $y = f(x) + \eta$, for which $E[\eta] = 0$ and $\text{var}(\eta) = \sigma^2$, answer the questions below:

1.1 Question 1 a)

Proof that the expected MSE of the prediction of a LS regression model will be the sum of bias and variance. Hint: You should start by expanding $E[(y - f(x))^2]$. [You will find it helpful to familiarize yourself with bias-variance decomposition in Section 3-2 of Bishop's book:

The MSE is: $E[(y_{\text{real}} - y_{\text{predict}})^2]$, consider $E[(y - f(x))^2] = E[(f(x) + \text{noise} - f(x))^2] = E[(\text{noise})^2]$, since variance of noise is $\sigma^2 = E[(\text{noise})^2 - E[(\text{noise})]] = E[(\text{noise})^2 - 0] = E[(\text{noise})^2]$, therefore $E[(y - f(x))^2] = E[(\text{noise})^2] = \sigma^2$.

So the MSE is $E[(y_{\text{real}} - y_{\text{predict}})^2] = E[(f(x) + \text{noise} - y_{\text{predict}})^2] = E[(f(x))^2 + 2 * f(x) * \text{noise} + \text{noise}^2 - 2 * f(x) * y_{\text{predict}} - 2 * \text{noise} * y_{\text{predict}} + y_{\text{predict}}^2] = E[(f(x) - y_{\text{predict}})^2 + 2 * \text{noise} * (f(x) - y_{\text{predict}}) + \text{noise}^2] = E[(f(x) - y_{\text{predict}})^2] + E[2 * \text{noise} * (f(x) - y_{\text{predict}})] + E[\text{noise}^2]$, note that $E[\text{noise}^2] = \sigma^2$ and $E[\text{noise}] = 0$, therefore $\text{MSE} = E[(f(x) - y_{\text{predict}})^2] + E[2 * \text{noise} * (f(x) - y_{\text{predict}})] + E[\text{noise}^2] = E[(f(x) - y_{\text{predict}})^2] + \sigma^2$

Consider: Bias = $(E[f(x)] - E[y_{\text{predict}}])$, Variance = $E[(y_{\text{predict}} - E[y_{\text{predict}}])^2]$, we can reckon $E[(f(x) - y_{\text{predict}})^2] = (E[f(x)] - E[y_{\text{predict}}])^2 + E[(y_{\text{predict}} - E[y_{\text{predict}}])^2] = \text{Bias}^2 + \text{Variance}$, so $\text{MSE} = E[(f(x) - y_{\text{predict}})^2] + \sigma^2 = \text{Bias}^2 + \text{Variance} + \sigma^2$, The sum of bias and variance.

1.2 Question 1 b)

Briefly, discuss the concepts of under-fitting and over-fitting in terms of bias and variance. E.g., how do you expect the relative magnitudes of bias and variance to coincide with underfitting and/or overfitting.

Under-fitting: the model does not learn enough from the training data to capture the basic relationships of the data, in terms of bias and variance, the under-fitting model would tend to have high bias (the model oversimplifies the problem) and low variance (the model gives similar prediction results for different training data sets);

Over-fitting: the model fits too well to the given data set and also reckons the noise and outliers in the data as sth important, not just the expected output relationships. In terms of bias and variance, the over-fitting model is expected to have low bias (the model fits the training data well) and high variance (model is very sensitive to small changes in the training data, and behaves very differently on other training data, producing a poor result).

1.3 Question 1 c)

Intuitively, discuss the relation between bias and variance with respect to data size. For example, as the dataset size grows very large, how do you expect bias and variance to behave?

Bias is caused by differences between the underlying assumptions of the model and the distribution of the actual data, and the value of bias would roughly remain the same when dataset size grows very large, if we do not change our assumption within the model.

Variance refers to how inconsistent a model's predictions are towards one training data set and to another. As the dataset size grows very large, the model has more data to learn from and is therefore less sensitive to random fluctuations or noise in the training data, which means that the model will behave more consistently across different training datasets, resulting in less variance.

2 Question 2: Principal Component Analysis

Beyond PCA's role in dimensionality reduction and visualization, the PCA algorithm finds extensive utility across various domains, such as image processing. For example, PCA has been employed for image rotation, where it assists in aligning images and correcting for orientation variations. This application is particularly valuable in fields of medical imaging where precise alignment of images is crucial for analysis and visualization purposes.

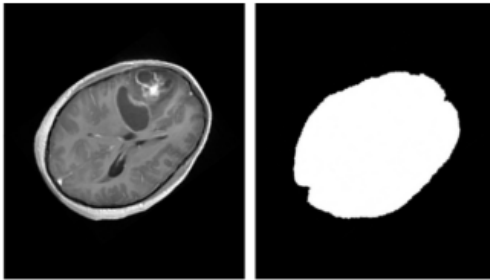


Figure 1: MRI image and its corresponding masked image.

2.1 Question 2.1)

Figure 1 displays a magnetic resonance image (MRI) alongside its corresponding masked version. In this context, a mask is a binary image that highlights areas of interest with white pixels, while the rest of the image remains black. Creating such masks can be accomplished through various methods, including manual annotation and automated segmentation techniques such as intensity thresholds, texture analysis, machine learning algorithms, or a combination thereof. The masked image isolates regions of interest from the background, enabling us to concentrate specifically on the relevant areas for our analysis. In this question intuitively discuss on employing PCA with the masked image to determine the rotation matrix required to align the MRI image with the Y-axis.

we can align the MRI image with the Y-axis using PCA and masked image by:

1. Apply PCA on masked image to find the main components and principal axis of the region of interest in the mask image, which usually indicates the main direction in the image.
2. Determine a rotation method (matrix, function, etc) by gained main components: Since we identified the main axis of the image, we can develop a rotation matrix that rotates the image from the current direction to the direction aligned with the Y-axis by focusing on the first principal component (the vector representing the most important direction of change in the image) and the Y-axis.
3. Rotates the MRI: Once we get the rotation matrix, we can apply it to the original MRI image so that the main structure of the image is aligned with the Y-axis.

2.2 Question 2.2)

Missing data refers to the absence of values in a dataset that can occur due to various reasons, such as equipment malfunction, human error, or simply because certain information was not collected. In such cases, PCA can help in data completion or filling the missing values.

To formulate this, let's assume we have images with d pixels represented as 2D arrays. Now suppose that we rearrange the pixels of each image into a single column vector (e.g., by stacking the columns of the array on top of each other). Let's denote N such images as column vectors $\{y_i\}_{i=1}^N$ where each y_i in $\mathbb{R}^{d \times 1}$. And suppose we use PCA to find a low-dimensional representation of the images, given by $x = W^T y$, where the matrix W is a $d \times K$ matrix. maps an image vector y into the K -dimensional subspace, the coefficients of which are x . As discussed in the notes, the columns of W are the K principal eigenvectors of the covariance matrix of the image data.

Given a point x in $\mathbb{R}^{K \times 1}$ in the subspace, we can reconstruct the corresponding image using $Y = Wx$. PCA learns the mapping by trying to minimize the approximation error between the original data and the reconstructed images.

But suppose we are given just a subset of the pixels in an image. Perhaps we only see the right half of the image, or we only see every other row of the image. It turns out that you can still find a subspace representation for this partially observed image, and in doing so, then reconstruct the image to fill in the missing pixels.

Your job will be to formulate this task, i.e., to formulate the task of filling in the unobserved (masked) part of an image. To begin, let's model the partial observation with a matrix that projects out the pixels we do not observe. Let $\hat{y} = Py$, where $\hat{y}$ only includes the observed pixels of y . Here P in $\mathbb{R}^{m \times d}$ is a projection operator to a subspace of m observed pixels. You can think of P as comprising just M rows of the $d \times d$ identity matrix, so it selects only a subset of dimensions. Please answer the following:

2.3 Question 2 a)

Assume that images are vectorized (flattened into columns) in row-major order. Then, when the bottom half of the image is missing, show that $P = [I_m | 0]$, where I_m is an identity matrix.

The purpose of the projection matrix P is to convert the complete image vector into a new vector containing only the observed pixels, excluding the bottom half missing image. During this conversion, P does this by keeping the pixels in the top half (through the I_m part) and discarding the pixels in the bottom half (0 part). Since $\hat{y} = P*y$, where $\hat{y}$ only includes the observed pixels of y , and P is a projection operator to a subspace of m observed pixels, then we can see that P does 2 things:

1. copy the "observed" part of y , which is done by multiplying the wanted vector section in y by an identity matrix I_m ;
 2. discard/delete/ignore the "missing" part of y , which is done by multiply that part of the vector by 0.
- having 2 things combined, we can derive that $P = [I_m | 0]$, where I_m is an identity matrix.

2.4 Question 2 b)

Now, given a partially observed image $\tilde{y}$, derive the optimal subspace projection of the image that minimizes the squared reconstruction error, given by $\|PWx - \tilde{y}\|^2$.

To find the optimal subspace projection x such that it minimize the squared reconstruction error $\|PWx - \tilde{y}\|^2$, we first rephrase $\|PWx - \tilde{y}\|^2 = (PWx - \tilde{y})^T * (PWx - \tilde{y})$ and find its derivation: $2PW^T PWx - 2PW^T \tilde{y} = 0$, solve the equation and we get:

$PW^T PWx = PW^T \tilde{y}$, we get the optimal subspace projection of the image that minimizes the squared reconstruction error $x = (PW^T PW)^{-1} PW^T \tilde{y}$

3 Question 3: Clustering

In lectures we discussed the use of the K-Means algorithm to cluster data into K number of groups (see Section 16.1), for which we used Euclidean distance to measure the proximity of each data point to the centroid of its respective cluster.

3.1 Question 3.1

When dealing with a dataset featuring varying scales among its features, normalization (or feature reweighting) is critical to prevent any single feature from becoming dominant. Normalization seeks to equalize the importance of each feature by transforming them to exhibit a distribution with a mean of zero and a standard deviation of one. Derive a mathematical expression to incorporate feature normalization into the K-Means objective function.

To normalize features st their mean=0 and covariance = 1, we can normalize each k th feature of data point i : $y_{i,k}$ by: $y'_{i,k} = \frac{y_{i,k} - \mu_k}{\sigma_k}$, where μ_k is the mean and σ_k^2 is the variance of feature k among all data points, and we develop new cluster centers c'_j out of normalized data $y'_{i,k}$. define $L'_{i,j} = 1$ if y'_i assigned to c'_j , 0 otherwise. The mathematical expression to incorporate feature normalization into K-Means objective function is done by sub in $y'_{i,k}$, c'_j and $L'_{i,j}$ into original objective function: $E'(\{c'_j\}_{j=1}^K, L) = \sum_{i,j} L'_{i,j} \|y'_i - c'_j\|^2$, where y'_i and c'_j are the normalized data points and cluster centers.

3.2 Question 3.2

In addition to the Euclidean distance, alternative distance functions such as L1 and cosine distance can be utilized within the K-Means objective function. L1 distance, also known as Manhattan distance, is a measure of the distance calculated by summing the absolute differences between the coordinates of corresponding points. In other words,

it's the sum of the absolute differences of the Cartesian coordinates. For instance, in two dimensions, the L1 distance between points (x_1, y_1) and (x_2, y_2) is given by $|x_1 - x_2| + |y_1 - y_2|$. Cosine distance measures the angular distance between two vectors in a multidimensional space. It's calculated as the cosine of the angle between the vectors. The formula for cosine similarity between two vectors u and v is given by: $\text{cosine similarity}(u, v) = 1 - \frac{u \cdot v}{\|u\| \|v\|}$, where $u \cdot v$ denotes the dot product of u and v , and $\|u\| \|v\|$ are the Euclidean norms (magnitudes) of vectors u and v respectively. Derive the cosine distance into the K-Means objective function.

$\text{cosine distance} = 1 - \text{cosine similarity} = 1 - \frac{u \cdot v}{\|u\| \|v\|}$.

The original K-Means objective function is: $E(\{c_j\}_{j=1}^K, L) = \sum_{i,j} \ell_{i,j} \|y_i - c_j\|^2$, where distance is $\|y_i - c_j\|^2$, now we replace distance with cosine distance, the new objective function is: $E'(\{c_j\}_{j=1}^K, L) = \sum_{i,j} \ell_{i,j} * (1 - \frac{y_i \cdot c_j}{\|y_i\| \|c_j\|})$

3.3 Question 3.3

Let's suppose that numerous features within the K-Means algorithm are redundant. For instance, there exists a significant correlation among features, potentially leading the K-Means algorithm to favor highly correlated features. What recommendations do you propose to enhance the model's performance?

We may take following methods to enhance the model's performance:

1. Reduce features to analysis by applying PCA: we can use PCA to abstract the main components of the data, reducing the workload it takes to iterate/develop a model.
2. We may also apply Hierarchical K-means to split the entire data into several data set and apply k-means on those sets to reduce redundancy's effect on all data but constrain them within the sets.
3. To avoid over-fitting of the model due to redundant data, we may add a punishing weight parameter into the model to avoid repeated data contributes too much on model's specific evolution.

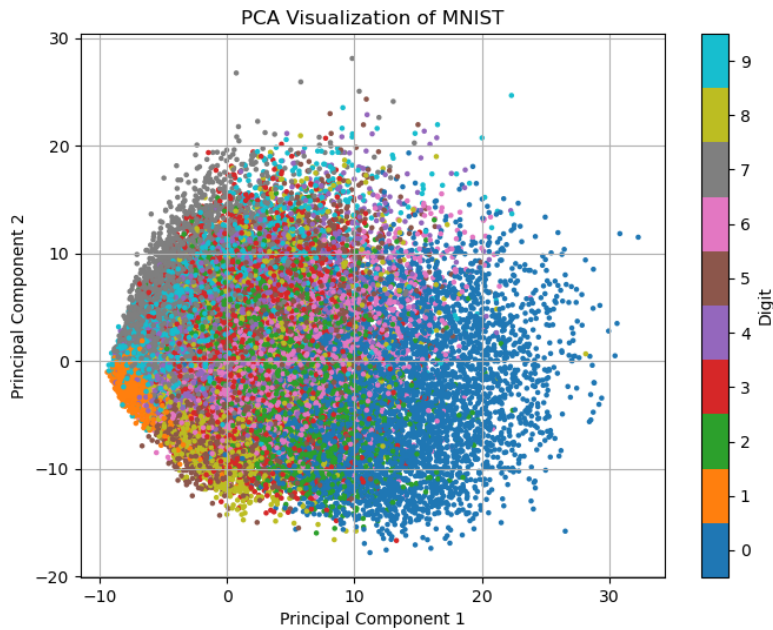
Programming Questions

4 Question 1. Dimensionality reduction and visualization

For this programming section, we will use the MNIST1. As we discussed before, the dataset comprising images of handwritten digits, centered and resized to 28×28 pixels. Each image has been saved as a vector with the dimension of $784 = 28 \times 28$, and is associated with a label, namely, digits 0 through 9. We also used previously introduced machine learning library for python programming language, scikit-learn, often referred to as sklearn. Please complete the code based on the text provided for each section in the Python file. The blank sections that need completion are indicated by XXX. The completed python files do not need to be submitted.

4.1 Question 1.1

As we discussed, each digit in the dataset can be represented as a 784-dimensional vector, but visualizing data in 784 dimensions is not practically feasible. Here, we aim to reduce this to 2 dimensions for visualization purposes. We will utilize the first section of the PCA.py script to accomplish this task. After filling in the necessary sections of the script, you should be able to generate a 2D plot of the data. Upon examining the plot, discuss whether we can identify distinct clusters within the data. Do you expect different runs of applying PCA results in the same output? Why?

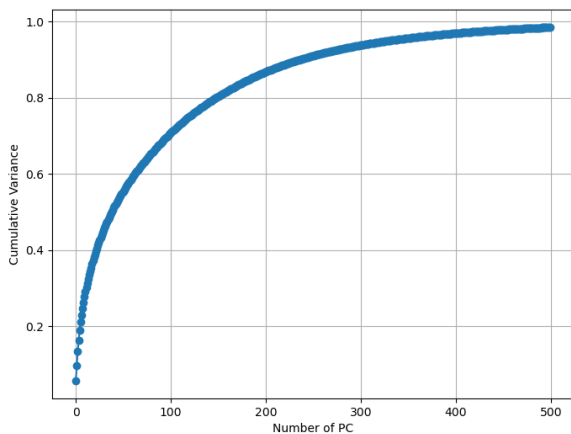


After applying PCA dimensionality reduction, we can observe data points clustered into different clusters in the generated 2D graphs, which represent different handwritten digits(blue cluster, red cluster, yellow cluster, etc.). Since PCA tries to preserve maximum variance, similar numbers will tend to cluster together in the two main components.

We can expect different runs of applying PCA results in the same output, since according to PCA.py, the PCA algorithm is always initialized the same, using the same method to select the initial principal component and the data set does not change, therefore the result should be the same every time the PCA is runned.

4.2 Question 1.2

In the second section of the PCA.py script, we will plot the cumulative variance explained by adding more principal components. Determine the number of components needed to capture 90% of the variance in the data.

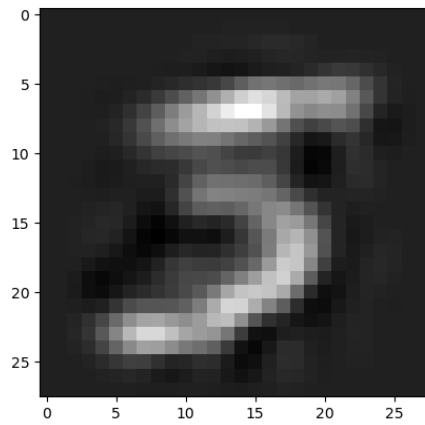


According to the generated graph, we can see that the number of components needed to capture 9/10 of the variance in the data is approximately 245.

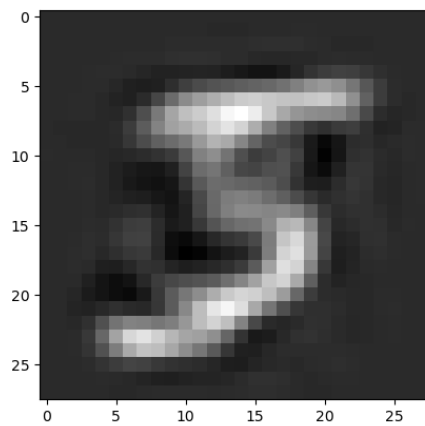
4.3 Question 1.3

In this section, we will use the third section of the PCA.py script to reconstruct input images using the first 50,100, and 500 principal components. Visualize the output and discuss the properties of added components?

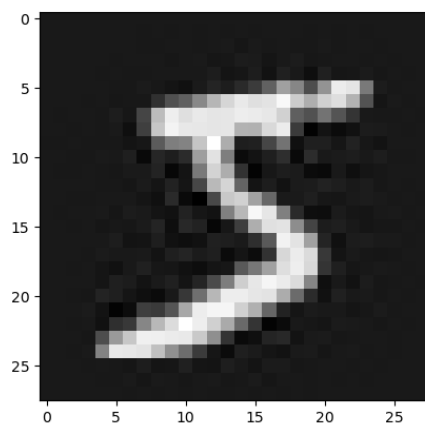
Component = 50:



Component = 100:



Component = 500:



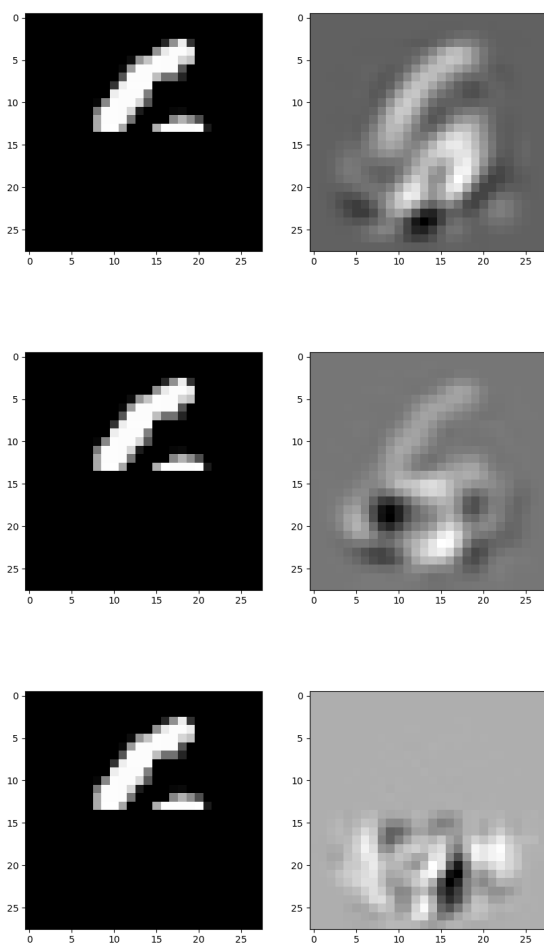
Properties of the added components: When component = 50, the result image presents a blurry picture because only a small fraction of the dimension of the data is captured. At this point, the image is a rough representation of

the shape of the number, with less detail, showing the initial effects of the reconstruction process; when component = 100, the clarity of the image is improved compared to using only 50 principal components, since more data is captured, making the contours and some details of the numbers (such as the "l" within 5 start to become more legible; when component = 500, the image appears sharper and richer in detail. Since most of the data is included, the reconstructed image is more accurately reflect the features of the original image. The shape, edges and internal details of the number are restored.

We can see that as the value of component increases, the result image becomes clearer and can distinguish the "5" written by hand. In PCA, principal components are extracted from the original data set to capture the main features of the original data. Therefore the more components added, the less the "lost data", and the image is more distinguishable.

4.4 Question 1.4

As discussed in theoretical question 2, PCA can be employed to address missing data. In this section, we introduce missing data by removing the bottom half of an image shown in Figure 2 Using the mathematics derived from 2.b, complete the fourth section of the PCA.py script to reconstruct the input image. Visualize the output and discuss how increasing the number of principal components changes the reconstructed image.



As we used 50, 100, and 500 principal components to reconstruct an image of a handwritten number "6" with the lower half removed, the results showed that the reconstructed image became more chaotic as the number of principal components increased. When component = 50, the reconstructed picture can roughly be distinguished as 6, but the circle in 6 became bigger when component = 100, and the result image become undistinguishable wiehnn component = 500.

When we introduce more principle component to be counted, the model not only captures major variances in the data but also starts to capture unimportant noise (messy, cluttered pixels within original picture) and mess up with the development of the correct model. Also when the component number grows too big, we might encounter overfitting and the model is attempting to reconstruct the number while also misinterpreting and reconstructing noise in the data, resulting in the reconstructed image looking more cluttered.

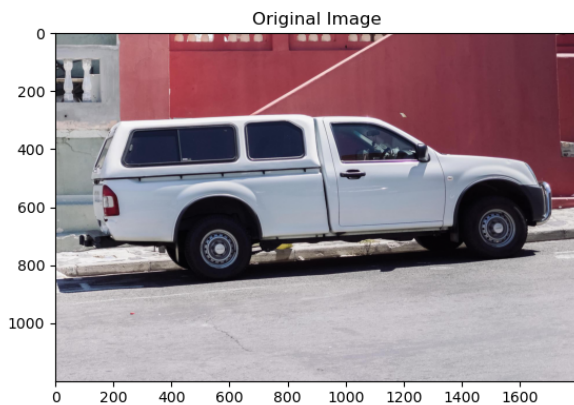
5 Question 2. Segmentation with Clustering

Image segmentation stands as a fundamental task in the computer vision, crucial for pinpointing objects and delineating boundaries within images. Essentially, image segmentation is the assignment of a label to each pixel in an image, grouping pixels with similar attributes together. This process finds extensive application in everyday scenarios like autonomous vehicles, facial recognition and detection, video surveillance, and satellite image analysis. A multitude of algorithms can be enlisted to accomplish the segmentation of regions of interest within input data. Previously, a simple approach involved leveraging clustering algorithms atop RGB images. This method entailed grouping pixels based on their color similarity, allowing for the identification and isolation of distinct regions within the image. In this question, we are looking to explore how to use KMeans and GMM clustering algorithm to segment an input image. We will use sklearn to perform clustering. Having this in mind, please answer below questions.

5.1 Question 2.1

We will use the first section in Clustering.py to explore the argument for the KMeans algorithm. We are looking to segment an input image into 3 segments. Execute the algorithm and display the segmented image. Report the cluster center. If we run the algorithm three times, will we obtain consistent results? (Note: you should compare the center of the cluster). Will we consistently receive the same labels each time we run the model? Discuss your observation.

The original picture is:



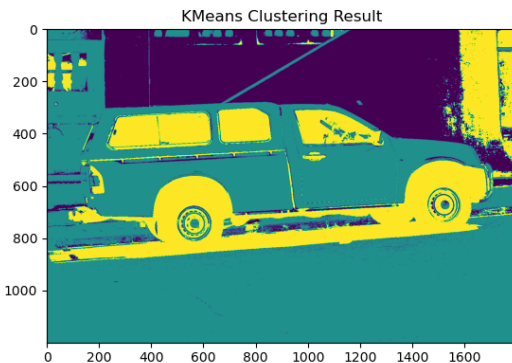
By using `print(kmeans.cluster_centers_)`, I get the cluster centers for each of executions during 3 run-time:

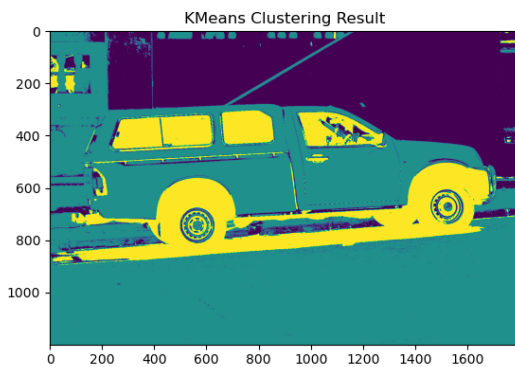
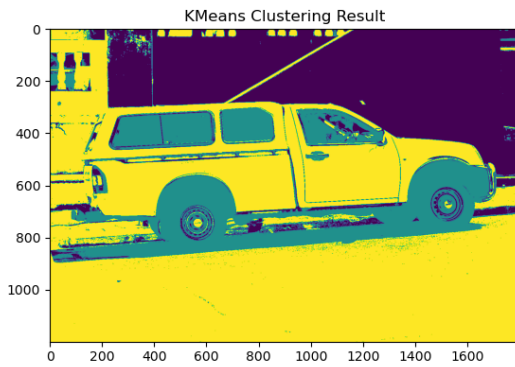
1st cluster centers: `[[0.50201315 0.25841242 0.30504925] [0.0883945 0.08545425 0.13049737] [0.73330868 0.72483125 0.74977238]]`

2nd cluster centers: `[[0.1696647 0.11370172 0.1608789 ] [0.73411347 0.72750682 0.75244757] [0.53911114 0.28225914 0.3269089 ]]`

3rd cluster centers: `[[0.50201315 0.25841242 0.30504925] [0.0883945 0.08545425 0.13049737] [0.73330868 0.72483125 0.74977238]]`

the result image:

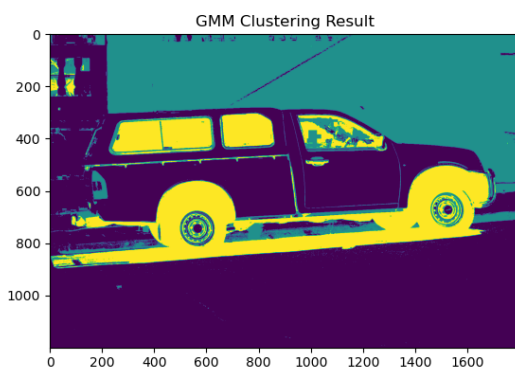




If we run the algorithm three times, because we randomly selecting the initial cluster center of the KMeans algorithm during initialization, the cluster center may be different each time the algorithm is run, thus even with the same input data, different runs may result in different clustering results. As shown in the result, we may obtain similar output in 1st and 3rd run, but the 2nd result shown in graph is entirely different from that we get in 1st and 3rd run.

5.2 Question 2.2

We will use the second section in Clusteirng.py to explore the argument for the GMM algorithm. We are interested to explore how full covariance shape to segment the input image into 3 segments. Show the results of the segmentation. Report the mean and covariance of each component.



by using `print(gmm.means_)` and `print(gmm.covariances_)`, we have:

Means (Cluster Centers) of GMM:

```
[[0.72959546 0.72817597 0.75420619]
 [0.51249323 0.27081747 0.31409667]
 [0.08539084 0.08397403 0.12992251]]
```

Covariances of GMM:

```
[[[0.01034863 0.00862341 0.00748354]  
[0.00862341 0.00759759 0.00689717]  
[0.00748354 0.00689717 0.00677471]]]
```

```
[[0.01981011 0.00837886 0.0075096 ]  
[0.00837886 0.01155624 0.00989568]  
[0.0075096 0.00989568 0.00903889]]]
```

```
[[0.00191286 0.00212362 0.00317897]  
[0.00212362 0.00254957 0.00398801]  
[0.00317897 0.00398801 0.00665537]]]
```